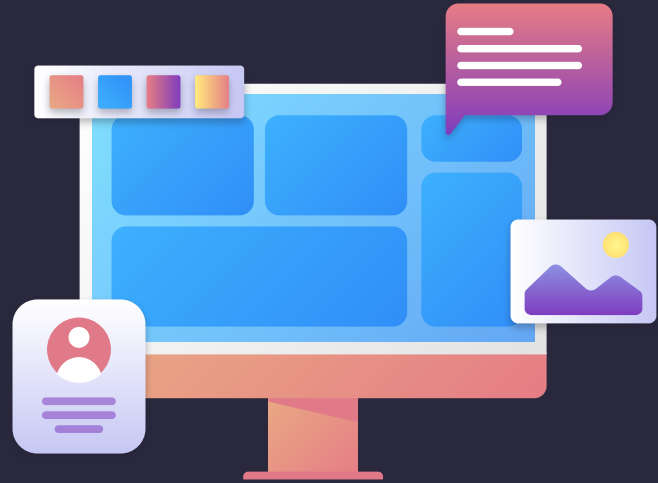
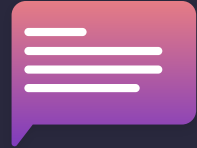


Tabela Hash

Resolução de Problemas
Estruturados em Computação



O QUE É UMA TABELA HASH



Também conhecida como **tabela de dispersão** ou **tabela de espalhamento**. É a estrutura

> de dados que associa chaves <
de pesquisa à valores, com
objetivo de buscar elementos
de forma eficiente.

Buscando: João

[0]	[1]	[2]	[3]	[4]
Ana	Pedro	João	Maria	Bruna

Normalmente quando queremos verificar se um determinado dado existe em uma estrutura, nós verificamos, a partir do início, índice por índice até que o dado seja encontrado, ou não.

Buscando: João

[0]	[1]	[2]	[3]	[4]
Ana	Pedro	João	Maria	Bruna

Normalmente quando queremos verificar se um determinado dado existe em uma estrutura, nós verificamos, a partir do início, índice por índice até que o dado seja encontrado, ou não.

Buscando: João

[0]	[1]	[2]	[3]	[4]
Ana	Pedro	João	Maria	Bruna

Normalmente quando queremos verificar se um determinado dado existe em uma estrutura, nós verificamos, a partir do início, índice por índice até que o dado seja encontrado, ou não.

Buscando: João

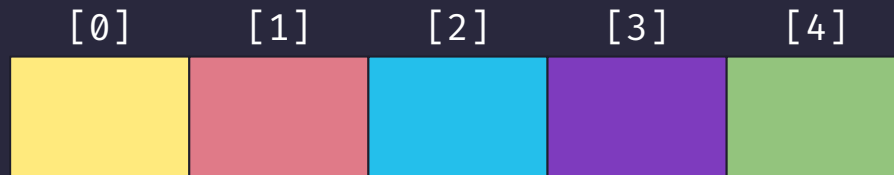
[0]	[1]	[2]	[3]	[4]
Ana	Pedro	João	Maria	Bruna

Esse tipo de busca é o que chamamos de **busca sequencial**. Não é considerada uma busca eficiente, pois no caso da estrutura possuir muitos dados, seriam necessárias muitas iterações para encontrar o valor desejado.

/TABELA HASH

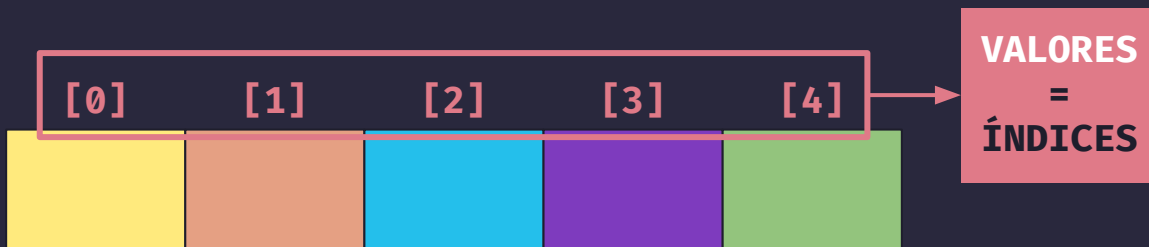


Vamos fazer isso utilizando um vetor



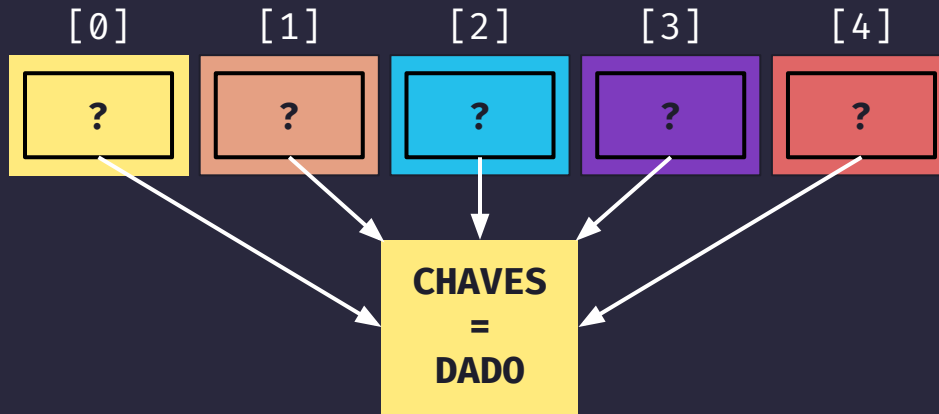
/TABELA HASH

O **valor** que vamos utilizar vão ser correspondentes aos índices do nosso vetor.
No caso o nosso valor é um inteiro que vai de 0, até o N-1 (tamanho do vetor - 1)



/TABELA HASH

A **chave** é o dado que nós vamos inserir dentro deste vetor. Essa chave pode ser **int**, **float**, **boolean**, **String**, **etc.**



/TABELA HASH

Vamos transformar essa **chave** em um **valor** que vá de 0 até N-1 (tamanho do vetor - 1)

FUNÇÃO HASH



Também conhecida como **função de espalhamento** ou **função de mapeamento**. Transforma uma **chave** em um **valor** de 0 a N-1.

/EXEMPLO DE INSERÇÃO COM FUNÇÃO HASH

CHAVES

Ana
Pedro
João

FUNÇÃO HASH

Quantidade de Letras - 1

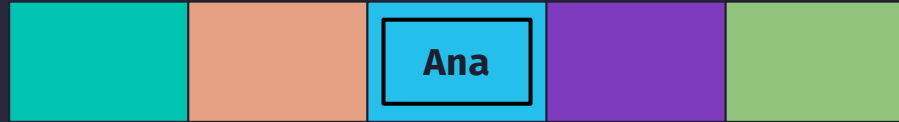
[0]

[1]

[2]

[3]

[4]



CHAVE = ANA

Quantidade de Letras = 3

FUNÇÃO HASH: $3 - 1 = 2$

VALOR = 2

/EXEMPLO DE INSERÇÃO COM FUNÇÃO HASH

CHAVES

Ana
Pedro
João

FUNÇÃO HASH

Quantidade de Letras - 1

[0]

[1]

[2]

[3]

[4]



CHAVE = PEDRO

Quantidade de Letras = 5

FUNÇÃO HASH: $5 - 1 = 4$

VALOR = 4

/EXEMPLO DE INSERÇÃO COM FUNÇÃO HASH

CHAVES

Ana
Pedro
João

FUNÇÃO HASH

Quantidade de Letras - 1

[0]

[1]

[2]

[3]

[4]



CHAVE = JOÃO

Quantidade de Letras = 4

FUNÇÃO HASH: $4 - 1 = 3$

VALOR = 3

Buscando um valor em uma TABELA HASH

A busca em uma tabela hash é feita pela **chave** que foi inserida. Para isso, ao invés de utilizarmos a busca sequencial, nós iremos utilizar a mesma **função hash** que utilizamos na inserção.

/EXEMPLO DE BUSCA COM FUNÇÃO HASH

CHAVE DE BUSCA

Pedro

FUNÇÃO HASH

Quantidade de Letras - 1

[0]

[1]

[2]

[3]

[4]

Ana

João

Pedro

CHAVE = PEDRO

Quantidade de Letras = 5

FUNÇÃO HASH: $5 - 1 = 4$

VALOR DA CHAVE BUSCADA = 4

🌟 QUE PERFEITO 🌟

Um mundo onde podemos inserir e
buscar por chaves diretamente sem
precisar fazer uma busca sequencial
infinita pelos valores. É a disney
das estruturas de dados!!!



 EIS QUE... 

COLISÃO



/EXEMPLO DE COLISÃO COM FUNÇÃO HASH

INSERIR CHAVE

Maria

FUNÇÃO HASH

Quantidade de Letras - 1

[0]

[1]

[2]

[3]

[4]



CHAVE = MARIA

Quantidade de Letras = 5

FUNÇÃO HASH: $5 - 1 = 4$

VALOR = 4

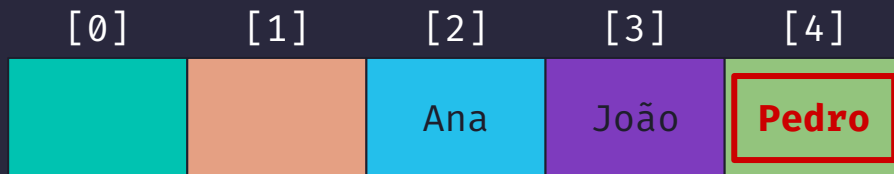
/EXEMPLO DE COLISÃO COM FUNÇÃO HASH

[0]	[1]	[2]	[3]	[4]
		Ana	João	Pedro

CHAVE = MARIA

Aqui temos um caso típico de **colisão** em uma tabela hash, quando duas **chaves** resultam em um mesmo **valor**.

/EXEMPLO DE COLISÃO COM FUNÇÃO HASH



CHAVE = MARIA

Mas e agora? Qual seria a solução?
Jogo Pedro fora e coloco a Maria no lugar? Descarto a Maria e deixo o Pedro? Mas e quando eu for buscar um deles e não encontrar, não vai ter problema?

/EXEMPLO DE COLISÃO COM FUNÇÃO HASH

[0]	[1]	[2]	[3]	[4]
		Ana	João	Pedro

CHAVE = MARIA

Claro que vamos manter os dois valores!

COLISÕES



Existem diversas formas para realizar o tratamento de colisões nas tabelas hash.

Duas formas que vamos estudar nesta disciplina:

1. **Encadeamento exterior**
(separado)
2. **Endereçamento aberto**
(interior)

/ENCADEAMENTO EXTERIOR (separado)

As chaves que, ao serem inseridas, colidem com outra chave, são armazenadas fora do vetor utilizado pela tabela Hash.

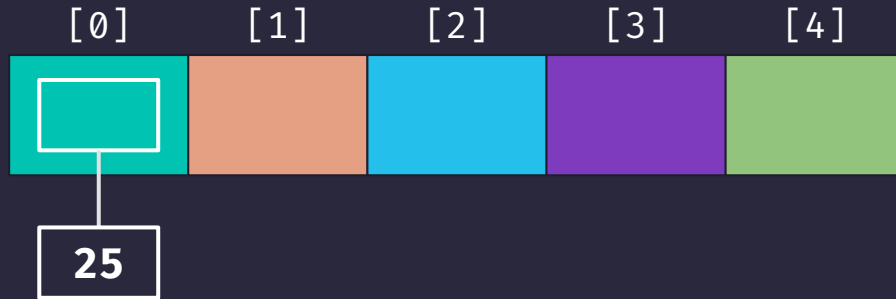
Isso acontece por meio de:

Listas Encadeadas

Para cada chave que será inserida na tabela, cria-se uma lista encadeada na posição onde a chave será inserida. Dessa forma já preparamos o nosso algoritmo para futuras colisões.

/ENCADEAMENTO EXTERIOR - INSERÇÃO

Inserindo chave 25



FUNÇÃO HASH

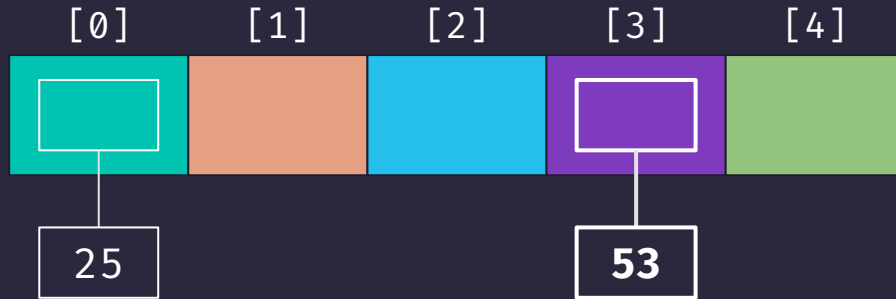
Chave % Tamanho

$$25 \% 5 = 0$$

Ao invés de inserirmos o 25 direto na posição 0, criamos uma lista encadeada e inserimos o 25 na primeira posição.

/ENCADEAMENTO EXTERIOR - INSERÇÃO

Inserindo chave 53



FUNÇÃO HASH

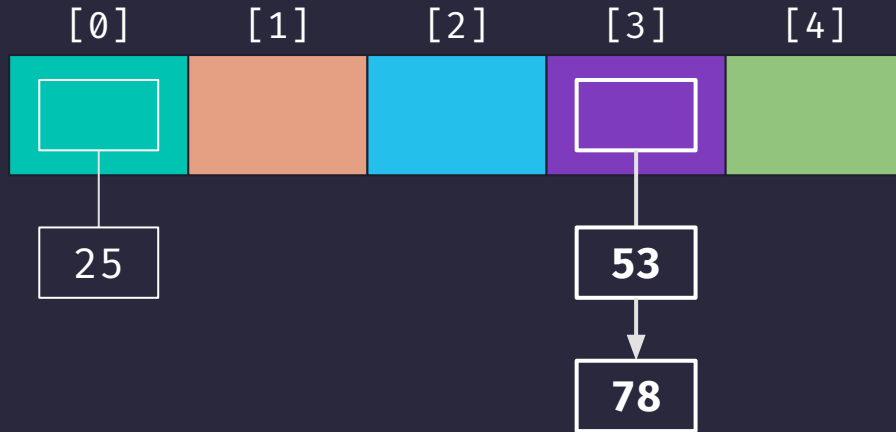
Chave % Tamanho

$$53 \% 5 = 3$$

Ao invés de inserirmos o 53 direto na posição 3, criamos uma lista encadeada e inserimos o 53 na primeira posição.

/ENCADEAMENTO EXTERIOR - INSERÇÃO

Inserindo chave 78



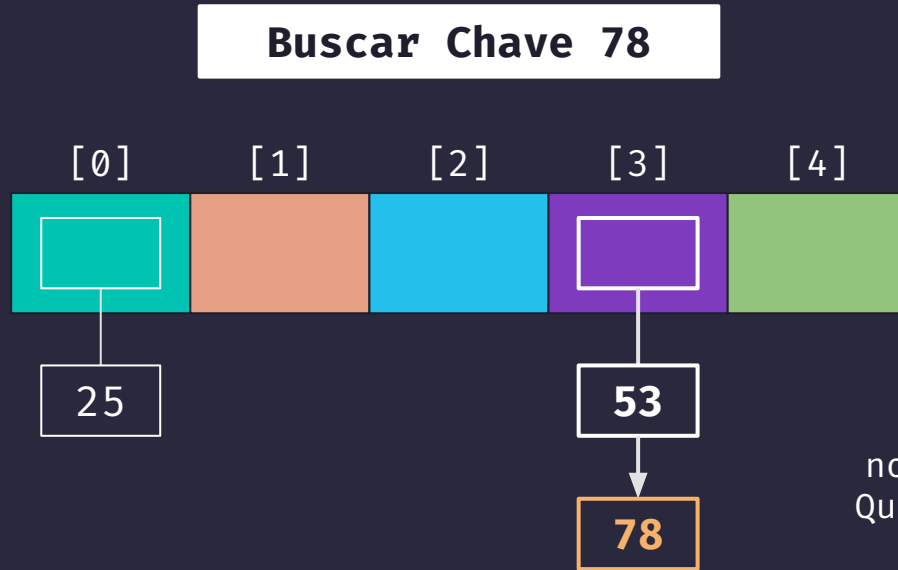
FUNÇÃO HASH

Chave % Tamanho

$$78 \% 5 = 3$$

Nesse caso temos uma ocorrência de colisão. O que fazemos agora é inserir o valor 78 na próxima posição da lista encadeada da posição 3.

/ENCADEAMENTO EXTERIOR - BUSCA



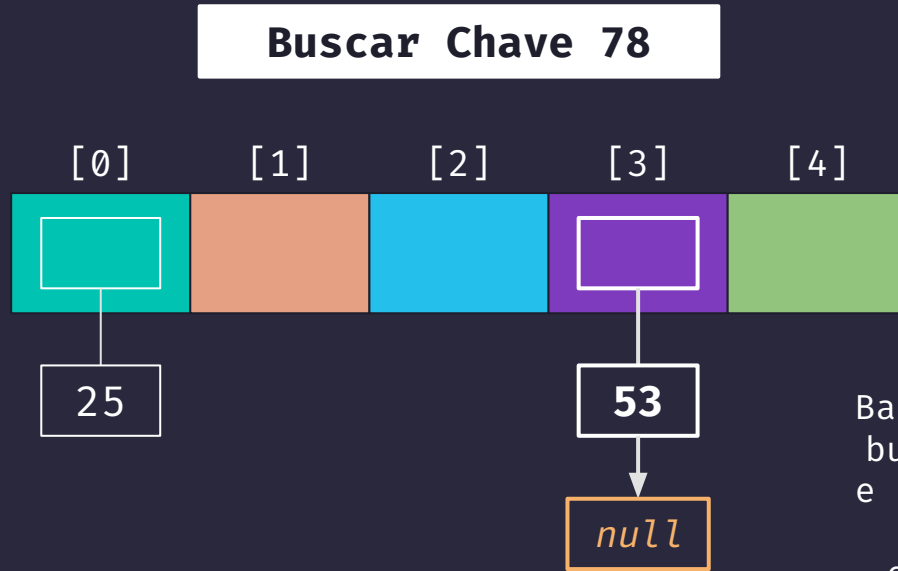
FUNÇÃO HASH

Chave % Tamanho

$$78 \% 5 = 3$$

Para buscar vamos aplicar a nossa função hash normalmente. Quando encontramos a posição no vetor, realizamos uma busca sequencial na lista encadeada.

/ENCADEAMENTO EXTERIOR - REMOÇÃO



FUNÇÃO HASH

Chave % Tamanho

$$78 \% 5 = 3$$

Basicamente o mesmo processo de busca. Aplicamos a **função hash** e realizamos a busca sequencial pela lista. Ao encontrar o elemento, basta removê-lo da lista encadeada.

/ENCADEAMENTO EXTERIOR - PROS E CONS

VANTAGENS

- Facilidade de implementação
- Tamanho flexível
- Busca eficiente em caso de poucas colisões
- Remoção direta

DESVANTAGENS

- Maior consumo de memória
- Baixa performance em caso de muitas colisões
- Acesso mais lento
- Reestruturação e redimensionamento

Encadeamento Exterior

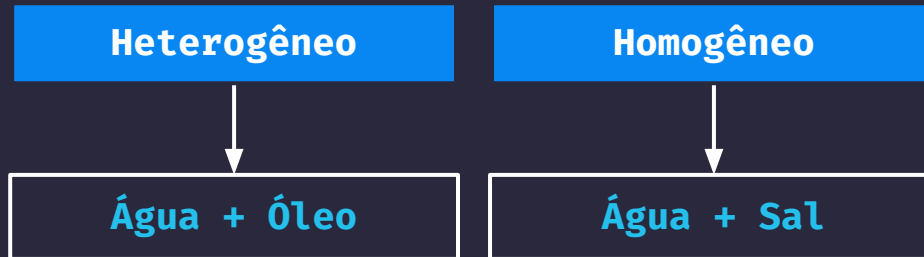
O PIOR CASO POSSÍVEL

O pior caso que pode
acontecer ao utilizar esta
abordagem é quando nossa
função hash ocasiona em
> muitas colisões, isso faz com <
que nossa tabela hash se
torne praticamente uma lista
encadeada

/ENDEREÇAMENTO ABERTO (interior)

Ao contrário do encadeamento exterior, no encadeamento interior as chaves que, ao serem inseridas, colidem com outra chave, são armazenadas no mesmo vetor utilizado pela tabela hash.

Podemos implementar o encadeamento interior de duas formas:



/ENDEREÇAMENTO ABERTO - DEFINIÇÕES

HETEROGÊNEO

No endereçamento aberto heterogêneo, as colisões são resolvidas **no mesmo vetor utilizado pela tabela**, mas em posições específicas separadas para o armazenamento de chaves que sofreram colisão.

HOMOGÊNEO

No endereçamento aberto homogêneo, todas as colisões são resolvidas **dentro da própria tabela**, ou seja, os elementos que colidem permanecem dentro da tabela hash, e a função de sondagem é usada para encontrar novas posições.

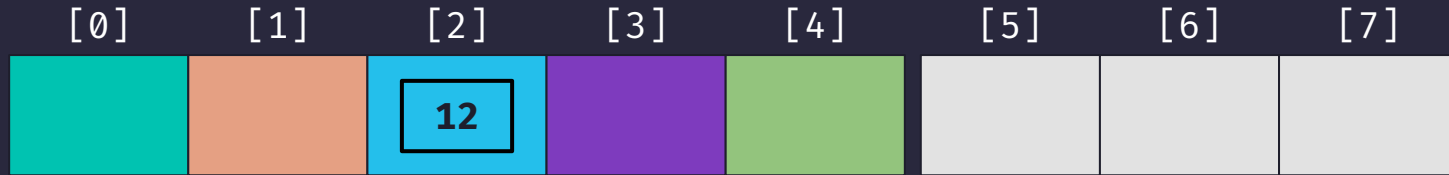
/ENDEREÇAMENTO ABERTO - HETEROGÊNEO



Nessa abordagem vamos utilizar um único vetor, porém um espaço ficará dedicado para **chaves** inseridas em espaços vazios (0 - 4), e o restante será dedicado para **chaves** inseridas em espaços ocupados (colisão).

/ENDEREÇAMENTO ABERTO - INSERÇÃO

Inserindo chave 12



FUNÇÃO HASH

Chave % Tamanho

$$12 \% 5 = 2$$

Perceba que a nossa função hash ainda considera o tamanho = 5, pois as demais posições são dedicadas exclusivamente à colisões.

/ENDEREÇAMENTO ABERTO - INSERÇÃO

Inserindo chave 41

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12					

FUNÇÃO HASH

Chave % Tamanho

$$41 \% 5 = 1$$

Perceba que a nossa função hash ainda considera o tamanho = 5, pois as demais posições são dedicadas exclusivamente à colisões.

/ENDEREÇAMENTO ABERTO - INSERÇÃO

Inserindo chave 37

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37		

FUNÇÃO HASH

Chave % Tamanho

$$37 \% 5 = 2$$

Aqui temos o primeiro caso de colisão. O que acontece nesse caso é: ao tentar inserir o 37 na posição 2, o algoritmo identifica que a posição já está ocupada, então insere o elemento na primeira posição disponível do espaço reservado para colisões.

/ENDEREÇAMENTO ABERTO - INSERÇÃO

Inserindo chave 51

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37	51	

FUNÇÃO HASH

Chave % Tamanho

$$51 \% 5 = 1$$

Aqui temos o segundo caso de colisão. O que acontece nesse caso é: ao tentar inserir o 51 na posição 1, o algoritmo identifica que a posição já está ocupada, então insere o elemento na primeira posição disponível do espaço reservado para colisões, que nesse segundo caso é inserido na posição seguinte ao 37.

/ENDEREÇAMENTO ABERTO - BUSCA

Buscando chave 37

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37	51	

FUNÇÃO HASH

Chave % Tamanho

$$37 \% 5 = 2$$

Utilizamos nossa função hash para encontrar o valor buscado. Ao verificar, o algoritmo identifica que o valor não foi encontrado na posição de saída da função (2).

/ENDEREÇAMENTO ABERTO - BUSCA

Buscando chave 37

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37	51	

FUNÇÃO HASH

Chave % Tamanho

$$37 \% 5 = 2$$

Com isso, o programa executa uma busca sequencial a partir da posição da área de colisão (5). A busca é realizada até que a chave seja encontrada, ou até encontrar uma posição vazia.

/ENDEREÇAMENTO ABERTO - REMOÇÃO

Removendo chave 51

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37	51	

FUNÇÃO HASH

Chave % Tamanho

$$51 \% 5 = 1$$

O caso de remoção é sempre muito semelhante ao de busca. Primeiro obtemos o valor da chave com a nossa função hash. Quando não encontramos o valor na posição, vamos procurar na área de colisões.

/ENDEREÇAMENTO ABERTO - REMOÇÃO

Removendo chave 51

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	41	12			37	<i>null</i>	

FUNÇÃO HASH

Chave % Tamanho

$$51 \% 5 = 1$$

Realizamos uma busca sequencial a partir da posição inicial da área de colisão (5), então removemos o valor desejado, caso este seja encontrado.

/ENDEREÇAMENTO ABERTO - REMOÇÃO

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
<i>null</i>	41	12	<i>null</i>	<i>null</i>	37	<i>null</i>	22

Cuidado ao remover elementos!!

Nesse caso, após remover o valor 51, perceba que a posição 6 do vetor ficou definida como *null*, que é o mesmo caso que indica uma posição **vazia**.

Qual é o problema disso?

/ENDEREÇAMENTO ABERTO - REMOÇÃO

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
<i>null</i>	41	12	<i>null</i>	<i>null</i>	37	<i>null</i>	22

Cuidado ao remover elementos!!

Lembrem que o algoritmo busca o elemento **até encontrá-lo ou até encontrar uma posição vazia**. Ou seja, se eu quiser buscar o elemento 22, o meu código vai entender que ele não existe, pois a posição 6 está vazia. Em termos de algoritmo isso significa que não existe nada inserido após a posição 6. Então **cuidado!**

/ENDEREÇAMENTO ABERTO - REMOÇÃO

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
<i>null</i>	41	12	<i>null</i>	<i>null</i>	37	-1	22

Cuidado ao remover elementos!!

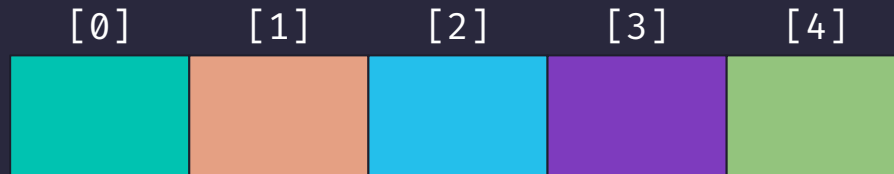
O que precisamos fazer na situação de **remoção**, é diferenciar a posição vazia que nunca teve um valor inserido, de uma posição vazia que teve um valor removido. Para isso, escolhemos um valor padrão para representar posições vazias por remoção. Um valor muito utilizado é o valor -1. Você pode deixar claro no código utilizando uma constante.

/ENDEREÇAMENTO ABERTO - HOMOGÊNEO



Nessa abordagem seguimos utilizando um único vetor. O que muda é que não temos um espaço dedicado para colisões. Vamos realizar um teste para encontrar a nova posição para **chaves** inseridas em espaços ocupados (colisão).

/ENDEREÇAMENTO ABERTO - HOMOGÊNEO



Nós iremos utilizar na disciplina a função de sondagem:

TESTE LINEAR

Outras funções mais complexas para você pesquisar (se quiser):

TESTE QUADRÁTICO

DOUBLE HASHING

REHASHING

RANDOM PROBING

/ENDEREÇAMENTO ABERTO - INSERÇÃO

Inserindo chave 26

[0]	[1]	[2]	[3]	[4]
	71	52	26	

FUNÇÃO HASH

Chave % Tamanho

$$26 \% 5 = 1$$

Em caso de colisão na inserção homogênea com teste linear, vamos inserir o valor na próxima posição disponível (posição 3).

/ENDEREÇAMENTO ABERTO - BUSCA

Buscando chave 26

[0]	[1]	[2]	[3]	[4]
	71	52	26	

FUNÇÃO HASH

Chave % Tamanho

$$26 \% 5 = 1$$

Primeiro obtemos o valor da chave que estamos buscando com a nossa função hash. Como não encontramos a chave no valor que deveria ter sido inserida, passamos a realizar uma busca linear a partir da próxima posição (2).

/ENDEREÇAMENTO ABERTO - BUSCA

Buscando chave 26

[0]	[1]	[2]	[3]	[4]
	71	52	26	

FUNÇÃO HASH

Chave % Tamanho

$$26 \% 5 = 1$$

A busca linear será executada até que a chave seja encontrada, ou até encontrar uma posição vazia, o que indica que aquela chave não foi inserida na tabela hash.

/ENDEREÇAMENTO ABERTO - REMOÇÃO

Removendo chave 26

[0]	[1]	[2]	[3]	[4]
	71	52	26	

FUNÇÃO HASH

Chave % Tamanho

$$26 \% 5 = 1$$

Aqui o processo é praticamente o mesmo da busca. Encontramos o valor da chave pela função hash, verificamos que a chave não é a que queremos remover, então passamos a realizar a busca linear a partir da próxima posição (2).

/ENDEREÇAMENTO ABERTO - REMOÇÃO

Removendo chave 26

[0]	[1]	[2]	[3]	[4]
null	71	52	-1	null

FUNÇÃO HASH

Chave % Tamanho

$$26 \% 5 = 1$$

Mas lembrem do caso que falamos na abordagem heterogênea, uma posição que nunca teve inserção e está vazia é diferente de uma posição vazia que já teve um valor. Portanto é importante diferenciar os valores para esses dois casos.

/ENDEREÇAMENTO ABERTO - COMPARATIVO

ASPECTO	HOMOGÊNEO	HETEROGÊNEO
Uso de Memória	Mais eficiente (todo o vetor é usado para dados)	Menos eficiente (espaço reservado pode ser subutilizado)
Complexidade de Implementação	Simples (uma única função de sondagem)	Mais complexo (necessário gerenciar áreas separadas)
Desempenho com Alta Carga	Performance degradada com alta carga	Melhor desempenho em cenários com muitas colisões
Clusterização	Suscetível a clusterização	Reduz o problema de clusterização
Remoção de Elementos	Mais difícil, exige manutenção das cadeias de sondagem	Mais simples, já que colisões têm áreas específicas
Desempenho da Sondagem	Degradado com altas colisões	Melhora ao isolar colisões em áreas específicas

Independente da abordagem ou função de sondagem utilizada para tratar colisões, o mais importante de uma tabela hash é:

UMA BOA FUNÇÃO HASH !

Mas afinal de contas, o que é uma boa função hash?

/FUNÇÃO HASH



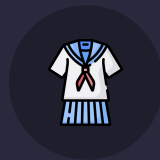
POUCAS COLISÕES

Quanto mais baixo for o número de colisões da sua função hash, melhor.



BAIXA COMPLEXIDADE

O cálculo precisa ser simples e eficiente. Não adianta ganhar tempo de busca se perdemos tempo calculando o valor.



SER UNIFORME

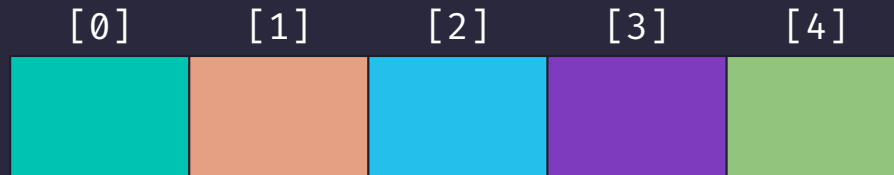
O tempo de execução do cálculo precisa ser constante para qualquer tipo de chave.

Outro ponto importante para levar em consideração para obter uma boa eficiência na sua tabela hash é:

FATOR DE CARGA

Mas o que é isso e como funciona?

/FATOR DE CARGA



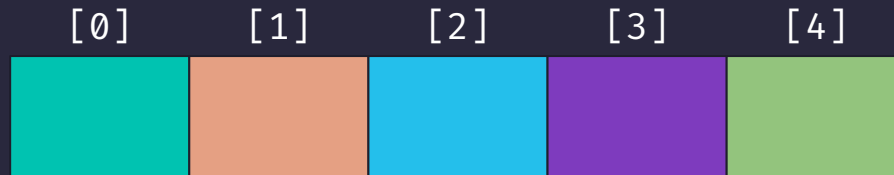
Fator de Carga = 0.75

Capacidade = 5

LIMITE DE INSERÇÃO = CAPACIDADE * FATOR DE CARGA

O fator de carga é um percentual estipulado para definir, juntamente à capacidade total da tabela, qual é o máximo de chaves que podem ser inseridas na tabela.

/FATOR DE CARGA



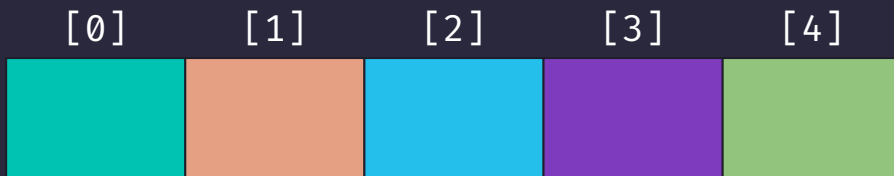
Fator de Carga = 0.75

Capacidade = 5

LIMITE DE INSERÇÃO = CAPACIDADE * FATOR DE CARGA

O ideal é manter um FC em aproximadamente 70%
Quanto mais próximo o FC for do tamanho, menos espaços disponíveis para armazenar chaves. Porém, quanto mais próximo do 0, mais espaço desnecessário alocado em memória.

/FATOR DE CARGA



Fator de Carga = 0.75

Capacidade = 5

LIMITE DE INSERÇÃO = CAPACIDADE * FATOR DE CARGA

Você pode utilizar esse cálculo para dobrar a capacidade de inserção da sua tabela hash. Mas tenha em mente que ao aumentar o tamanho do vetor da tabela hash, é preciso remapear (rehashing) os elementos já inseridos.